Calcolo parallelo e distribuito

Lezione del 05-10

Supercomputing -> Risolvere un problema usando un super computer

Application-depender -> La prestazione è misurata dal tempo necessario per risolvere una particolare applicazione

TOP 500 -> lista dei calcolatori più veloci al mondo

RMAX -> performance misurata dai benchmark di LINKPACK per la risoluzione di AX=B

Teraflop -> Flop -> operazioni floating point per secondo-> 10^{12} flops

TURNAROUND TIME -> come ridurre i tempi di risposta?

In generale:

una rappresentazione semplificata del tempo richiesto dalla risoluzione numerica di un problema è:

$$\tau = kT(N)\mu$$

- μ rappresenta un tempo di esecuzione (limitato per limiti teorici e tecnici che non possono essere superati)
- dove T(N) rappresenta complessità di tempo dell'algoritmo

La complessità di tempo dell'algoritmo di **Cramen** per la risoluzione di sistemi lineari in funzione di n (n equazioni, incognite)-> n!

a complessità di tempo dell'algoritmo di **Gauss** per la risoluzione di sistemi lineari in funzione di n (n equazioni, incognite)-> n^3

CALCOLO PARALLELO -> suddividiamo il problema in più problemi più piccoli che vengono svolti contemporaneamente su più calcolatori

La quantità che meglio esprime l'importanza della decomposizione del problema è lo **SCALE UP** :

$$\frac{T(n)}{(P * T\frac{n}{P})}$$

Tipi di parallelismo:

1. **Parallelismo temporale**
    1. Pipeline (Catena di montaggio (si prende come esempio il caso dell'ALU))
2. **Parallelismo spaziale**
    1. Si esegue la stessa operazione con più hardware (Nel caso di un calcolatore sono presenti più ALU)
3. **Parallelismo asincrono**

1. operazioni diverse vengono svolte in contemporanea (si agisce su dati diversi nello stesso momento -> presenza di più CPU)

TASSONOMIA DI FLYNN

SISD-> monoprocessore

SIMD-> single instruction multiple data ->array processor (parallelismo di tipo spaziale)

MIMD-> distribuited Memory-> Massive Parallel Processor (parallelismo di tipo asincrono)

Lezione del 07-10

Calcolatore distribuito:

- insieme di calcolatori sparsi anche dal punto di vista geografico nel mondo collegati attraverso la rete
- sistema di nodi collegati da reti geografiche

Calcolo parallelo -> sistema di nodi collegati da switch specializzati e dedicati

La differenza è nella rete di connessione

```
                            | Calcolo distribuito         | Calcolo
parallelo              |
                            |-----------------------------+-----------------
---------------|
                            | Obiettivo: Performance      | Ri-uso di
risorse esistenti    |
                            | Risorse di calcolo omogeneo | Risorse di
calcolo etereogeneo |
```

Progetto SCoPE

Cluster -> insieme di macchine collegate tra loro in modo da lavore come una macchina sola

Linguaggio -> C con libreria di MPI
Compilatore -> OPENMPI

I comandi andranno in modalità batch attraverso il sistema PBS

Per compilare sul cluster -> si scrive un file .pbs -> job-script che contiene una serie di direttive

Il codice inizia come un classico file bin/bash -> #!/bin/bash

Le righe che iniziano con #PBS sono direttive

Con flags:

```
-q -> in che coda mettersi

-l -> quanti core (<8)
```

```
-N -> job da eseguire

-o -> output e dove salvare

-e -> errori e dove salvarli
```

Ritornando alla riga di comando usiamo il comando qsub < nome del file >.pbs

A quel punto facciamo cat .err ed .out

Con qstat (visualizzo la coda) -> qstat -q studenti

Lezione del 12-10

MPI -> message passing Interface -> tra le implementazioni citiamo MPICH, OpenMPI (installata sul cluster)

Linguaggi ospiti:

- Fortran;
- C/C++;
- Matlab

Il concetto di MPI è che avendo a disposizione una macchina a memoria distribuita possiamo si può connettere più CPU contemporaneamente utilizzando memorie comuni

Si implementa un ambiente standard:

- in MPI un programma è visto come un insieme di componenti (o processi) concorrenti
  - questi verranno raggruppati e messi all'interno di un comunicator

Per comunicator intendiamo un gruppo di processori a cui vengono date delle regole per far si che comunichino tra di loro

Il comunicator supremo (**MPI_COMM_WORLD**) è il comunicator più potente, ma anche quello più anarchico (si crea di default)

MPI è una libreria che contiene:

- funzioni per definire l'ambiente;
- Funzioni per comunicare 1 a 1;
- funzioni per comunicazioni collettive;
- funzioni per operazioni collettive

Le funzioni dell'ambiente

```c
//un programma che stampa a video l'identificativo menum e il numero di processori nproc di tutti i procesori di MPI_COMM_WORLD
#include <stdio.h>
```

```c
#include "mpi.h"
int main (int args, char *argv[])
{
    int menum, nproc;

    MPI_Init(&argc,&argv); //inizializza l'ambiente di esecuzione MPI, MPI_COMM_WORLD, gli argomenti

    MPI_Comm_rank(MPI_COMM_WORLD,&menum); //identificativo

    MPI_Comm_size(MPI_COMM_WORLD,&nproc); //numero di processi totale

    printf("Sono %d di %d\n ", menum, nproc);

    MPI_Finalize(); //chiude l'ambiente del programma MPI, non si può più richiamare nessun'altra routine di MPI

    return 0;
}
```

Lezione del 14-10

Primi esempi:

Calcolo della somma di N numeri a_0 +...+ a_n

Su un calcolatore con un solo processore la somma è calcolata eseguendo le N-1 addizioni una per volta secondo un ordine prestabilito

L'algoritmo per un calcolatore parallelo->

l'idea è di dividere il problema di dimensione N in P problemi con dimensioni N/P e risolverli su più calcolatori

Somma (MIMD) -> Somme parziali

Delle soluzioni possibili possono essere:

- Ad ogni passo ciascun processore calcola la propria somma parziale (operazioni concorrenti) ed invia tale valore ad un unico processore prestabilito (operazione sequenziale)
- Un'altra soluzione può essere quella di fare le somme parziali a 2 a 2 e infine unirle su di un unico processore prestabilito

Per il primo caso il codice è di questo tipo

```
begin
forall P_{i}, 0  1 <= p-1 do
            sum := 0
```

```
            h:= i *(n/p)
            for j=h to h+(n/p) -1 do
                sum_{1} = sum_1} + a_{j}
            endfor
            if P_0 then
                for k = 1 to p-1 do
                    recv(sum_{k} , p_{k})
                    sumtot:= sumtot + sum_{k}
                endfor
            else if P_{i} then
                send(sum_i, P_{0})
            endif
            endif
        endforall
end
```

Il numero delle operazioni non è legato al numero di passi -> un calcolatore parallelo è in grado di eseguire più operazioni **concorrentemente** (allo stesso passo temporale)

Il tempo di esecuzione non è proporzionale alla complessità di tempo (non dipende soltanto dal numero di operazioni fl. p. effettuate)

La complessità di tempo non è adatta a misurare l'efficienza di un algoritmo parallelo

Con p processori ci aspettiamo che T(1) sia p volte T(p) -> \left(frac{T(1)}{T(p)}\right) = p -> ci aspettiamo di ridurre p volte il tempo di esecuzione

- **Speed up** -> Rapporto tra T_1 su T_p
- **Overhead** -> pT(p) - T_1 -> ciò che limita lo speed up ideale
- **Efficienza** -> E_p su p misura quanto l'algoritmo sfrutta il parallelismo del calcolatore

Efficienza ideale -> E^{ideale} (p) = $\frac{S^{ideale}(p)}{p}$ = 1

Lezione del 19-10

Comunicazione di un messaggio

Un dato che deve essere spedito o ricevuto attraverso un messaggio di MPI è descritto da:

(adress, count, datatype)

L'MPI presenta delle corrispondenze in C:

- MPI_INT (int);
- MPI_FLOAT (float);
- MPI_DOUBLW (double);
- MPI_CHAR (char);
- ...

La spedizione o la ricezione di un messaggio da parte di un processore può essere **bloccante** o **non bloccante**

Funzione bloccante:

MPI_Send

a cui corrisponde un **MPI_recv**
Presenta una variabile di tipo MPI_Status info

MPI_Send(&n, 1, MPI_INT, 1, tag, MPI_COMM_WORLD)

invia l'indirizzo n di tipo int e peso 1 (con indirizzo 1 ->a chi inviarlo) e lo invia con un tag impostato a MPI_COMM_WORLD. Per ricevere il tag deve essere uguale (tag.send = tag.recv):

MPI_recv(&n,1,MPI_INT,0,tag,MPI_COMM_WORLD,&info)

A differenza del send recv contiene anche il parametro info che conferma l'avvenuta ricezione.

Alla fine della comunicazione:

MPI_Get_count(&info, MPI_INT, &num)

Dove num rappresenta il numero di elementi ricevuti attraverso il parametro di tipo MPI_Status info

Analogamente per la funzione non bloccante troviamo un **MPI_Isend** a cui corrisponde un **MPI_Irecv**. Stesso ragionamento per quello bloccante con l'unica differenza è il parametro MPI_Request rqst, che contiene le informazioni sull'intera spedizione. Ha un ruolo simile a status, ma contiene più informazioni

Nel caso di un'operazione non bloccate consideriamo terminata quando nella spedizione il buffer è di nuovo utilizzabile dal programma, mentre nella ricezione invece quando l'operazione sarà completata.

Per conoscere se l'operazione sia terminata MPi mette a disposizione il controllo delle fasi come:

MPI_Test, MPI, Wait

I vantaggi delle operazioni non bloccanti sono i tempi di attesa minori e la possibilità di poter avere più operazioni contemporanee

Bisogna prestare attenzione ai problemi di incongrunze e sovrapposizione

Operazioni collettive

Queste tipi di operazioni sono eseguite da tutti i processori appartenti ad un comunicator

Può richiedere sincronizzazione ed è di tipo bloccante

Permettono di eseguire operazioni globali e di avere una gestione ottimizzata degli input/output seguendo uno schema ad albero.

- **MPI_Barrier** -> interrompe tutti i processi e li sincronizza (una volta arrivati tutti)
- **MPI_Bcast** (broadcast) -> invio di informazioni a tutti i processori -> a differenza della send Bcast indica solo chi deve inviare
- **MPI_Gather** -> tutti mandano e uno riceve -> quanti e cosa viene mandato + info per chi deve ricevere

- **MPI_Scatter** -> Identico a Gather con la differenza che uno soltanto invia

Lezione del 21-10

Peak performance:

- N_c è il massimo numero di operazioni floating point che possono eseguite dal processore in un'unità di tempo;
- T_c è il ciclo di clock.

La peak performance sarà uguale a:

$$P_{max} = \frac{N_c}{T_c} \; oppure \; P_{max} = N_c * f(frequenza)$$

Per prendere i tempi

```
#include <sys/time.h>
...
gettimeofday(...,...);
{blocco di istruzioni di cui vogliamo conoscere il tempo di esecuzione}
gettimeofday(...,...);
```

Lezione del 26-10

Per prendere i tempi:

1. Modificare il PBS
   a. PBS -l nodes=8:ppn=8
   b. sort -u $PBS_NODEFILE > hostlist
   c. NPCU='wc -l < hostlist'
   d. echo -----
   e. echo 'This job is allocated on ' ${NCPU}'cpu(s)''on hosts:'
   f. cat hostlist
   g. echo ----
2. Nel file C
   a. funzione double MPI_Wtime() (tempo in secondi)
   a. vanno dichiarati double t_{0}, t_{i} e t_{tot}
   b. Stampa del valore
   a. MPI_Reduce(&time, &timetot,1,MPI_DOUBLE,MPI_MAX,0,MPI_COMM_WORLD)

Lezione del 28-10

Valutazione dell'efficienza dell'algoritmo

Il numero delle operazionu non è legato al numero dei passi temporali. Il tempo di esecuzione non è proporzionale alla complessità di tempo (non dipende soltando dal numero di operazioni floating point)

L'overhead totale misura quanto lo speedup differisce da quello ideale

T(N) può essere diviso in due parti:

1. T_s relativo alle operazioni eseguite in maniera sequenziali;
2. T_c relativo alle operazioni che possono essere esguite concorrentemente;

Lezione del 04-11

ISOEFFICIENZA

$$T(1, n_1) = \left(\frac{p_1}{p_0}\right) T(1, n_0) < -T(1, n_1) = I(p_0,\ p_1,\ n_0)$$

La funzione I esprime la legge secondo cui deve aumentare la complessità di tempo dell'algoritmo sequenziale con dimensione n_1 su p_1 processori a partire da p_0 e da $T(1, n_0)$ affinchè l'efficienza scalata non degradi. I è detto **ISOEFFICIENZA**

Nel caso della somma nel passare da p_0 a p_1 con p_1 > p_0, la complessità di tempo del problema deve aumentare, rispetto alla dimensione iniziale n_0, del fattore

$$k = \frac{p_1\ log_2 p_1}{p_0\ log_2\ p_0}$$

Un algoritmo è scalabile se l'efficienza rimane costante (non degrada) al crescere del numero dei processori e della dimensione del problema

Su un calcolatore conta non solo il tempo di esecuzione (numero delle operazioni) ma anche il tempo impiegato nella comunicazione

Dato

- t_{com} tempo di comunicazione di un dato;
- t_{calc} tempo di esecuzione di 1 operazione;

Definiamo **overhead di comunicazione unitario** il rapporto

$$oc = \left(\frac{t_{com}}{t_{calc}}\right)$$

Lezione del 16-11

Memoria condivisa (OpenMp)

Con la memoria condivisa tutte le CPU possono accedere alla stessa cella di memoria. È in generale quindi opportuno sfruttare un parallelismo shared-memory

Threads -> unità di base di utilizzo della CPU

In questo modo si può pensare di dividere il lavoro su più threads -> abbiamo una maggiore velocità (si elimina il problema della comunicazione) ma è necessario un controllo per la sincronizzazione (visto

che ogni cpu può accedere alla stessa cella di memoria).
I thread vengono coordinati attraverso la sincronizzazione degli accessi alle variabili condivise (e possono essere creati dinamicamente)

Esempio di somma in P-like

```
begin
    sumtot:=0
    forall t_{i}, 0 <= i <= p-1 do
        s_{i} := 0
        h := i (n/p)
        for j = h to h +  (n/p) -1 do
            s_{i} := s_{i} + a_{j}
        endfor
        lock(sumtot)
        sumtot := sumtot + s_{i}
        unlock
    end forall
end
```

OpenMP -> Open specification for Multi Processibg (API per gestire il parallelismo shared-memory multi-threaded).

Molto portatile (Fortran, C/C++, UNIX, win). È composto da:

- Direttive per il compilatore (si usano per creare il team e per gesitre quali istruzioni devono essere eseguite in parallelo e altre in multi-thread)
- Runtime Library Routines, per intervenire sulle variabili di controllo interne allo standard run-time (includere il file omp.h)

Direttive:

```
#pragma omp directive-name [clause[[],]clause]...] new-line
{
    structured-block
}
clause: if(scalar-expression)
num_threads(integer-expression)
default(shared | none)
private(list)
firstprivate(list)
shared(list)
copyin(list)
reduction(operator: list)
```

Per compilare:

```
gcc -fopenmp -lgomp -o nome-eseguibile nome-codice.c
```

Da dare sempre attraverso il pbs

PBS:
Si modifica la vode del PBS -l nodes=1:ppn=8 e si aggiunge:

- export OMP_NUM_THREADS=2 (numero di thread generati di default durante le regioni parallele)
- export PSC_OMP_AFFINITY=TRUE (per legare i thread a particolari processori)
- gcc -fopenmp -lgomp -o $PBS_O_WORKDIR/somma $PBS_O_WORKDIR/sommaOpenMP.c

Lezione del 18-10

Prodotto Matrice-Vettore

La decomposizione dei dati del problema corrisponde ad una partizione in "blocchi" della matrice e del vettore. Il calcolo della soluzione viene ricondotto Al calcolo della soluzione relativa a ciascun blocco

Lezione del 23-11

MPI e le topologie

Se immaginiamo di disporre ad **anello** i processori abbiamo la limitazione di un collegamento diretto solo fra gli adiacenti; mentre sarà totale indirettamente
Un altra disposizione è la **griglia** si presenta una discrepanza per la comunicazione dei processori in base alla posizione (agli angoli e ai lati si presenta ha meno condivisioni dirette)
Per ovviare a questo problemo si introduce la topologia del **Toro**

In base al tipo di problema si sceglie un caso specifico di topologia

Una topologia è la geometria virtuale in cui si immaginano disposti i processori. La topologia virtuale non è limitata alla disposizione reale dei processori

Il caso di nostro interessse è la griglia

Lezione del 30-11

Approfondimenti

Progettazione di un algoritmo parallelo per architettura **MIMD** a memoria distribuita per il calcolo del prodotto di una matrice A per un vettore x:

$$Ax = y$$

$$A \in \ R^{nm}, con\ x, y \in \ R$$

Decomposizione del problema $\rightarrow$ prima si riformula l'algoritmo in blocchi, che vengono in qualche modo distribuiti ai processori.
I strategia

1. Decomposizione del problema
2. Risoluzioni dei sottoproblemi
    1. Da $Ax = y$ passiamo a p prodotti $\rightarrow \ A_i x = y_i$
3. Partizionamento della matrice in blocchi di righe

II strategia

1. Decomposizione del problema (divisione della matrice A in blocchi di colonne fra p processori, stessa cosa per il vettore)
2. Risoluzione dei sottoproblemi
    1. Da $Ax = y$ passiamo a p prodotti $\rightarrow \ A_i x_i = r_i$ dove

$$y = \sum_{i=0}^{P-1} (r_i)$$

3. Partizionamento della matrice in blocchi di colonne

III strategia
Divisione della matrice A per blocchi quadrati $\rightarrow$ Calcolo dei prodotti parziali. La comunicazione avviene ovviamente in parallelo

Lezione del 07-12

Calcolo Speed Up ed Efficienza

(prodotto Matrice-Vettore)

I° strategia:

$$Ax = b \; A \; \in R^{n \; x \; m} \; x \; \in \; R^m \; b \; \in \; R^n$$

- Distribuzione della matrice per righe (ad ogni processore va un insieme di righe intere)
- Il vettore X viene dato a tutti i processori
- Ogni processore calcola un prodotto matrice-vettore più piccolo

Dati:
p processi $n(\geq)p$

$$r = \frac{n}{p}$$

Avremo che

- $\frac{n}{p}$ se n % p = 0
- $\frac{n}{p} + 1$ se n % $p \neq \; 0$

$$T(1) = n(2m-1)t_{calc}$$
$$T(p) = r(2m-1)t_{calc}$$

Algoritmo sequenziale

```
for i = 0, n-1 do
    y_i = 0
    for j = 0, n-1 do
        y_i = y_i + a_ij x_j
    endfor
endfor
```

Per la strategia 1 lo Speed UP

$$S(p) = \frac{T(1)}{T(p)} = \frac{n}{p}$$

Nel caso in cui si volesse calcolare i passi di comunicazione avremo

$$T(p) = r(2m-1)t_{calc} + hr \; log_2 p t_{calc}$$

II° strategia:

- Distribuzione della matrice per colonne (ad ogni processore va un insieme di colonne intere)
- Il vettore X viene distribuito tra tutti i processori
- Ogni processore calcola un prodotto matrice-vettore più piccolo
- Richiede una ricompilazione del risultato

$$b = \sum_i s_i$$

Dati: q processi, $m(\geq)q$

$$c = \frac{m}{q}$$

Avremo che

- $\frac{m}{q}$ se n % p = 0
- $\frac{m}{q} + 1$ se n % $p \neq\ 0$

$$T(1) = n(2m-1)t_{calc}$$
$$T(q) = r(2c-1)t_{calc} + n\ log_2 q\ t_{calc}$$

Algoritmo a blocchi:

```
begin
    y = 0
    for i = 0 to p-1 do
        r_i = A_i x_i
        y = y + r_i
    endfor
end
```

Lo speedup nella II° stategia è

$$S(q) = \frac{T(1)}{T(q)} = \frac{n(2m-1)t_{calc}}{n(2c-1)t_{calc} + \ n(log_2 q\ t_{calc})}$$

III° strategia $\rightarrow$ processori distribuiti in una griglia p x q (la matrice è distribuita per blocchi ogni processore ha un blocco rettanglare)

Dati: p x q processi, $n(\geq)p$, $m(\geq)q$

$$r = \frac{n}{p}$$

Avremo che

- $\frac{n}{p}$ se n % p = 0
- $\frac{n}{p} + 1$ se n % $p \neq\ 0$

$$T(1) = n(2m-1)t_{calc}$$
$$T(p\ x\ q) = r(2c-1)t_{calc} + \ (r\ log_2\ q)\ t_{calc} + (r\ log_2\ q)\ t_{com}$$

$(r\ log_2\ q)\ t_{com}$ rappresenta l'albero per la riunione di vettori di r elementi in uno da n elementi, tra i processori di una colonna della griglia.

Lezione del 14-12

Lezione del 14-12

Architettura delle GPU

(cenni alla programmazione CUDA)

I sistemi attuali sono generalmente ibridi rispetto alla tassonomia di Flynn e molto spesso eterogenei.

Si è investito molto sull'elaborazione grafica (industria dei videogiochi) con lo scopo di alleggerire il carico computazionale della CPU

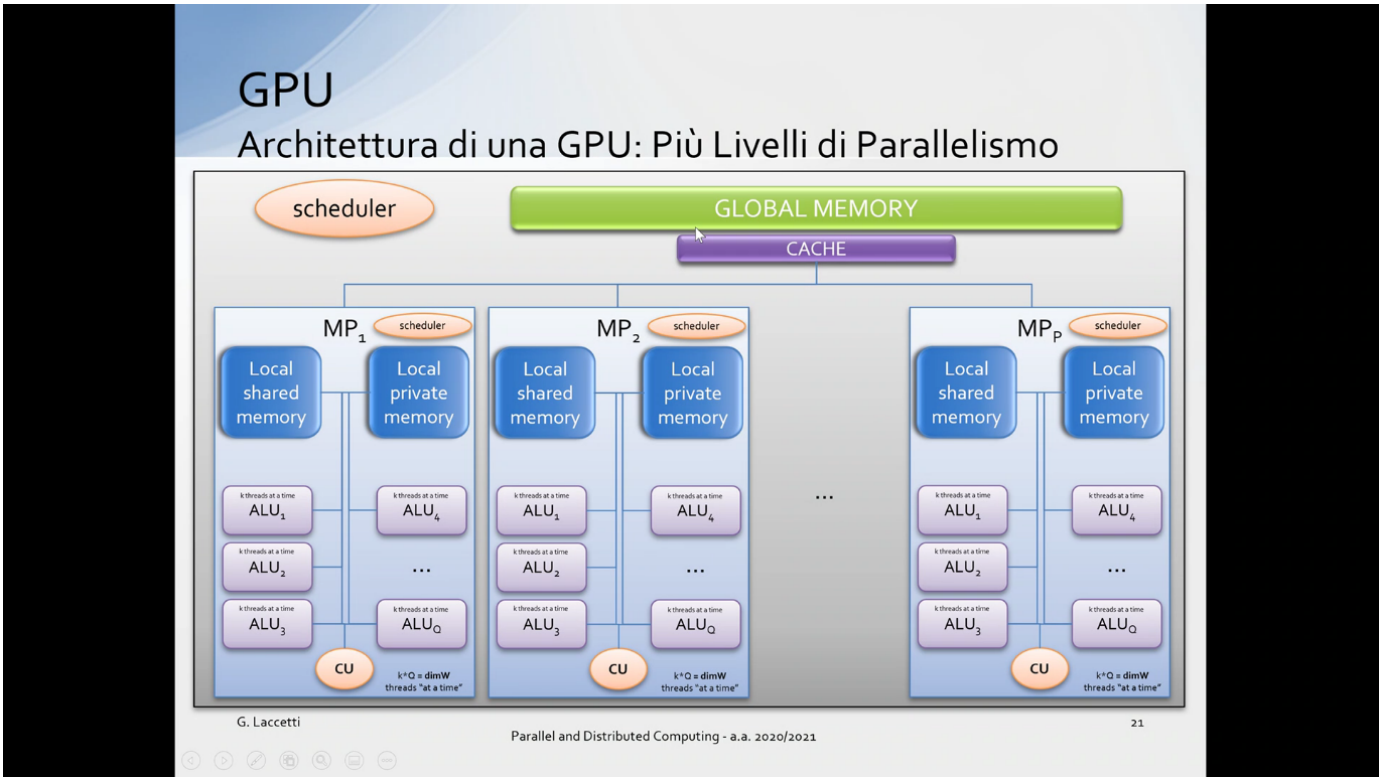
PIPELINE GRAFICA $\longrightarrow$ processo di rendering (generazione di un immagone a partire da una descrizione di ogetti 3D) realizzato attravero hardware più o meno specializzato

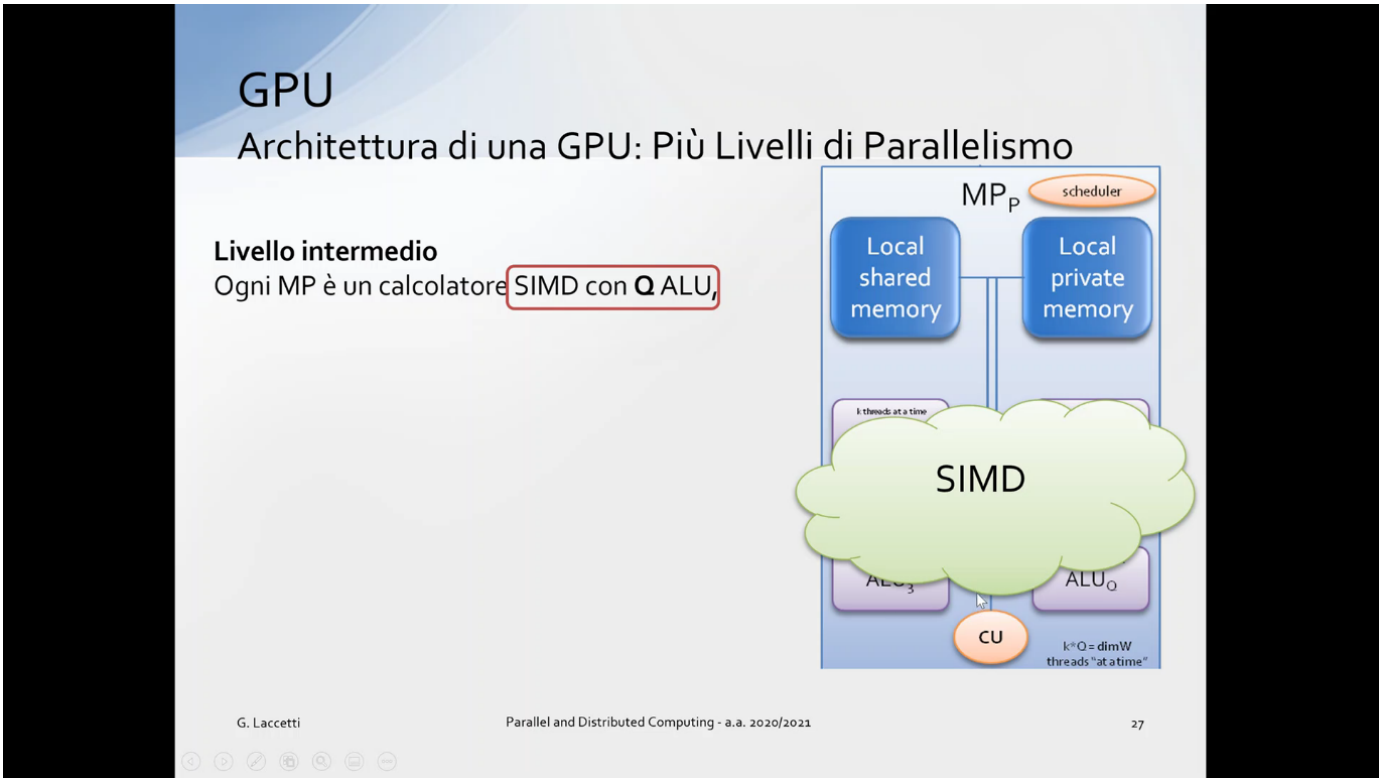
Bisogna vedere la GPU come un coprocessore (necessita però di una CPU per "vivere")

CUDA (*C*omputer *U*nified *D*evice *A*rchitecture) $\longrightarrow$ introdotta da NVIDIA, con lo scopo di creare un modello di programmazione parallela scalabile a memoria condivisa

A questa si accoda OpenCL (linguaggio concepito come open-source per rendere utilizzare CUDA con GPU non NVIDIA)

In generale un'architettura GPGPU è caratterizzata da più livelli di parallelizzazione

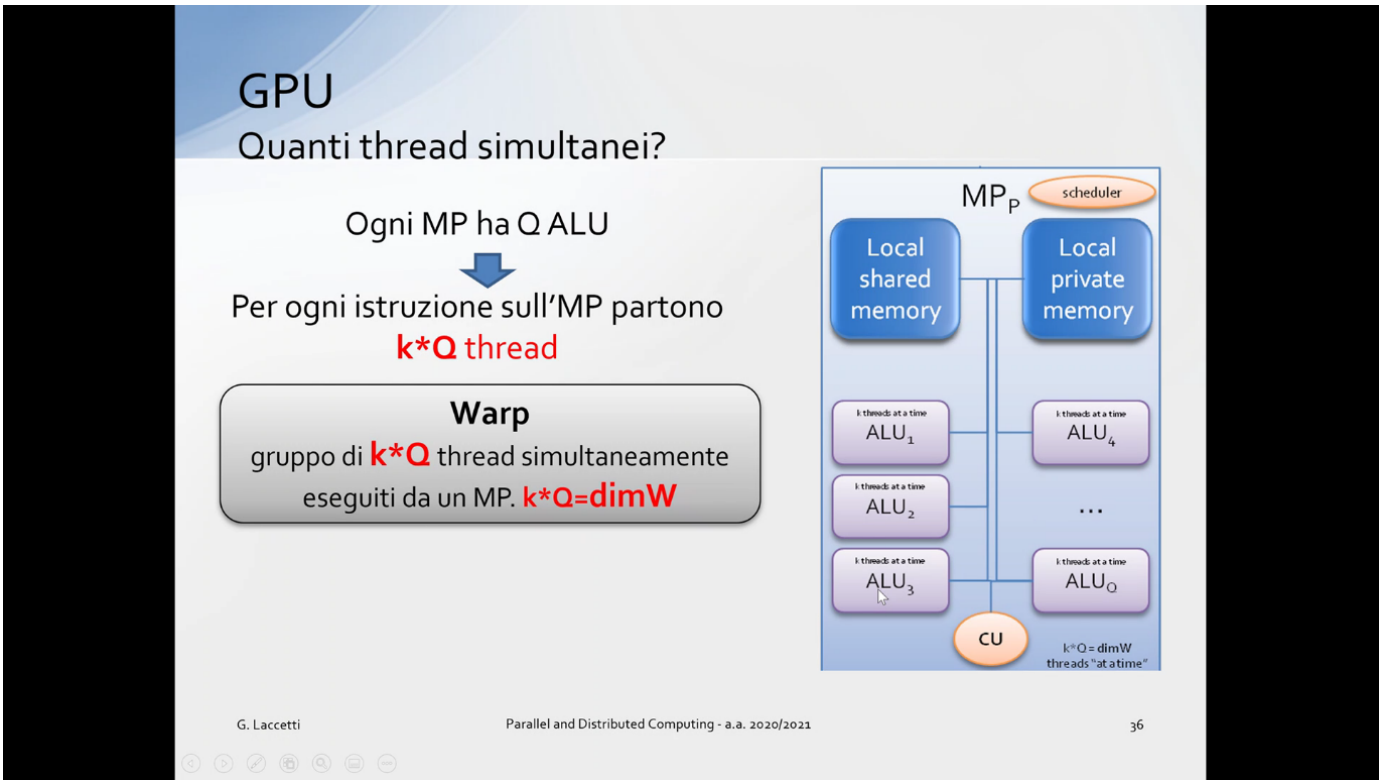




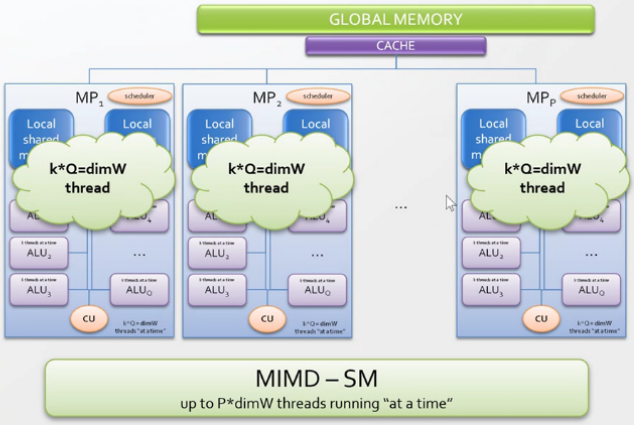
Livello più basso $\longrightarrow$ Ogni ALU è pipeline

Se consideriamo che operazione sia svolta da un thread diverso (1 operazione $\longrightarrow$ 1 thread) per ogni istruzione decodificata:

k thread eseguono l'operazione corrispondente su ogni ALU



GPU
Quanti thread simultanei?



Ogni GPU ha P MP

Per ogni istruzione sulla GPU partono fino a **P*dimW=P*k*Q** thread conteporaneamente

GPU
Tempo di calcolo

P core con Q unità
Pipeline con k segmenti
k thread per ogni unità

Ogni thread esegue istruzioni tutte dipendenti tra loro.

Algoritmo puramente parallelo
T_c tempo di esecuzione concorrente

IDEALE!!

Tempo d'esecuzione algoritmo per GPU

$$T_{GPU}(P \cdot Q \cdot k, N) = \frac{T_c(N)}{P \cdot Q \cdot k} \cdot t_{calc}$$

N dimensione del problema

A questo tempo va aggiunto quello di accesso alla memoria

Per compilarre in CUDA nella shell:

nvcc -o *nome-eseguibile* *nome-codice*.nu

Lezione del 16-12

Lezione del 16-12

Prodotto matrice matrice

Progettazione di un algoritmo parallelo per architettura MIMD a memoria distribuita per il calcolo di un prodotto matrice matrice

Prodotto Matrice-Matrice $AxB = C$ con $A, B \ \in R^{n\ x\ n}$
Nel caso di algoritmo sequenziale:

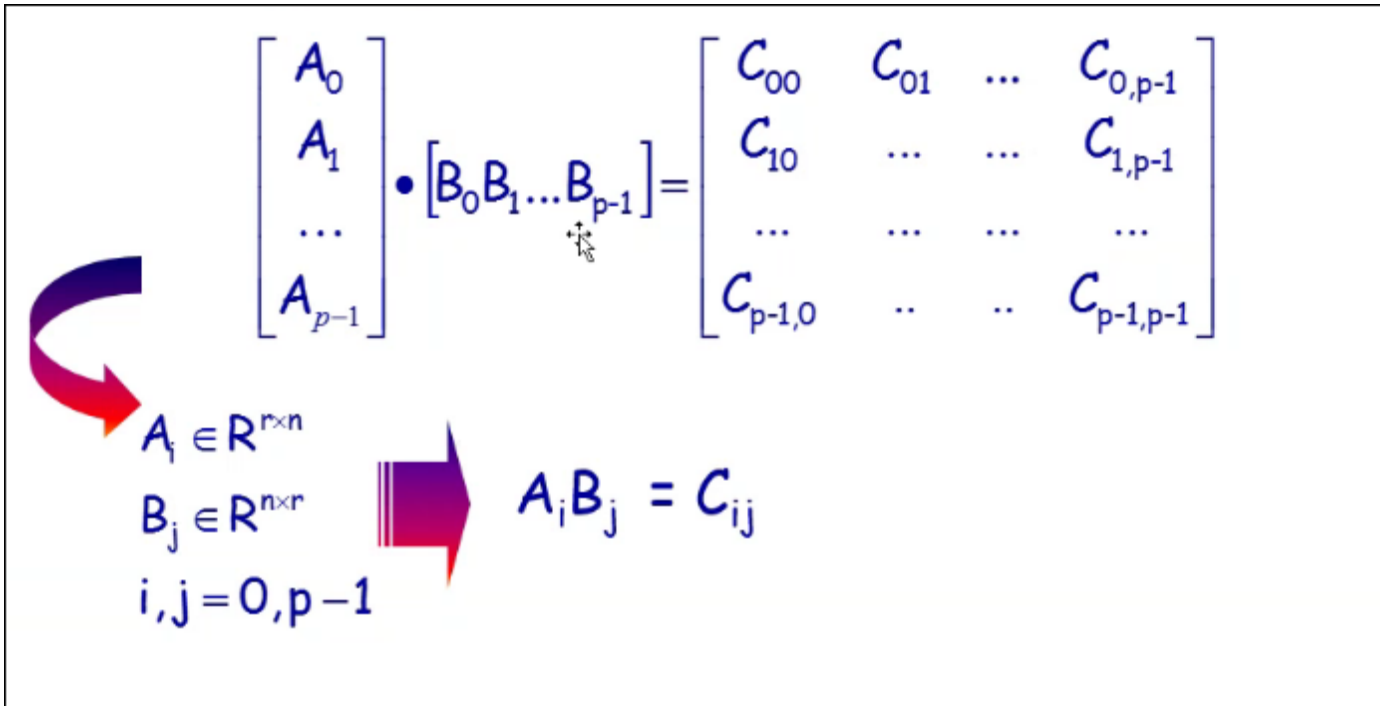
```
for i=0, n-1 do
        for j=0, n-1 do
        c_{ij} = 0
    for k = 0, n-1 do
```

$$c_{ij} = c_{ij} + a_{ik} b_{ik}$$

```
        endfor
    endfor
endfor
```

Il problema resta sempre il modo in cui gestire la distribuzione dei dati. Gli elementi di C sono calcolati effettuando i prodotti scalari di ciascuna riga di A e ciascuna colonna di B

Scegliamo quindi di sfruttare il problema a blocchi (***A*** per ***p*** blocchi di righe e ***B*** per ***p*** blocchi di colonne $\rightarrow$ Strategia 1)

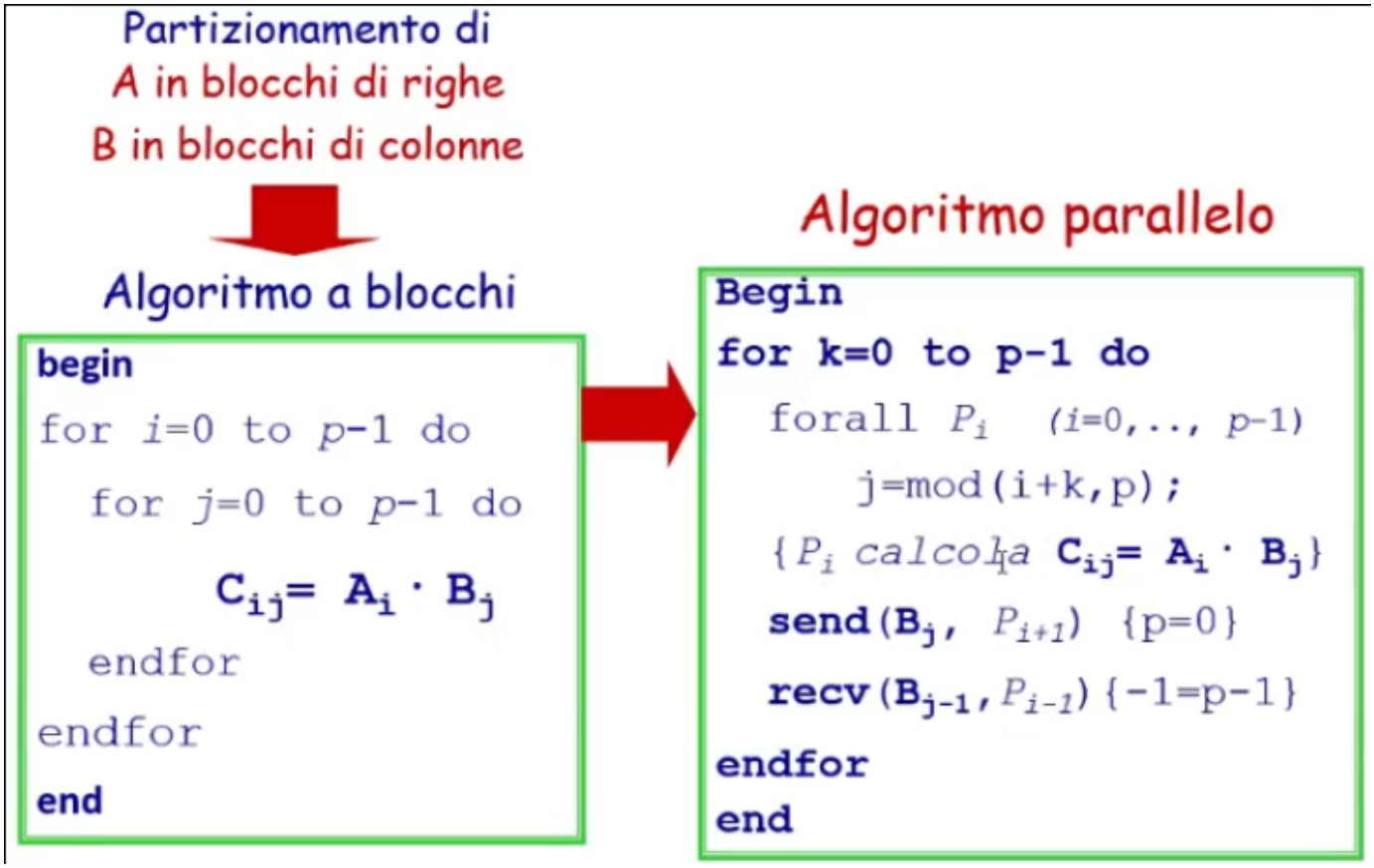


```
begin
    C = 0
    for i = 0 to p-1 do
    for j = 0 to p-1 do
```

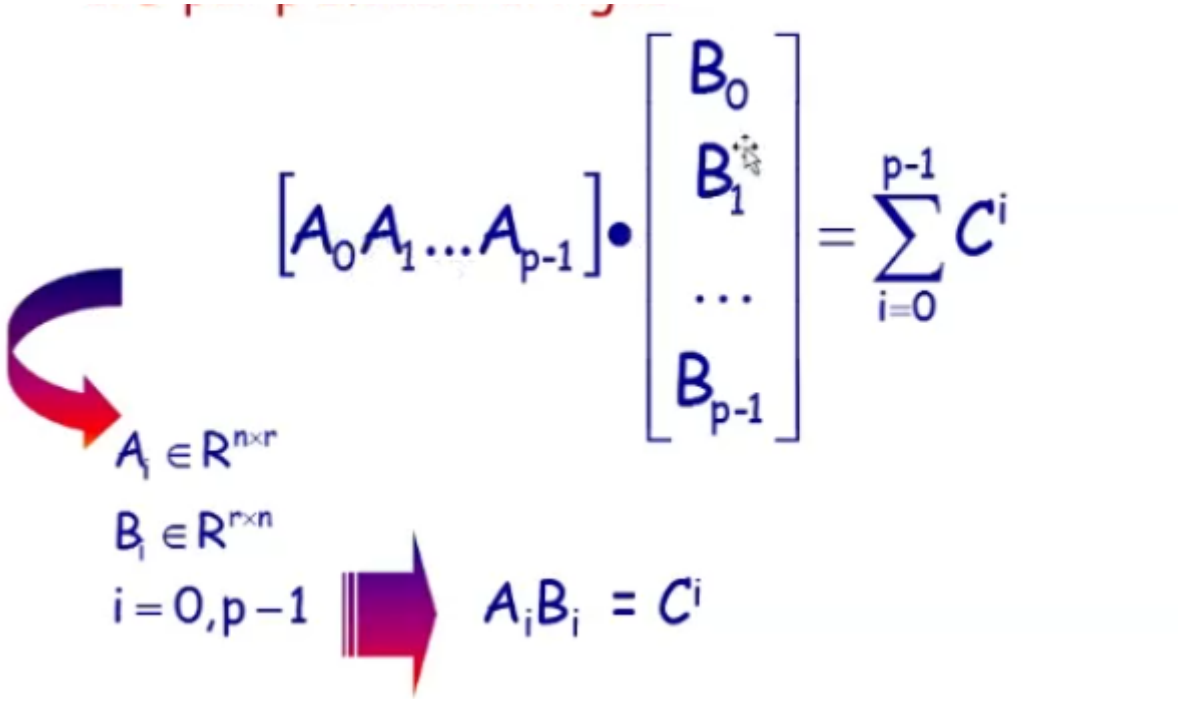
$$C_{ij} = A_i B_j$$

```
    endfor
    endfor
end
```

Calcoliamo prodotti matrice-matrice di dimensione minore



Una seconda possibilità potrebbe essere quella di suddividere la matrice ***A*** per ***p*** blocchi di colonne e la matrice ***B*** per ***p*** blocchi di righe



Distribuiamo tra i processori il calcolo dei diversi prodotti matrice-matrice ì. Distribuiamo tra i processori gli elementi delle 2 matrici

Inizialmente tutti i processori calcolano indipendentemente, un contributo parziale di tutte le componenti di C; successivamente in parallelo tutti i processori concorrono alla somma dei contributi parziali

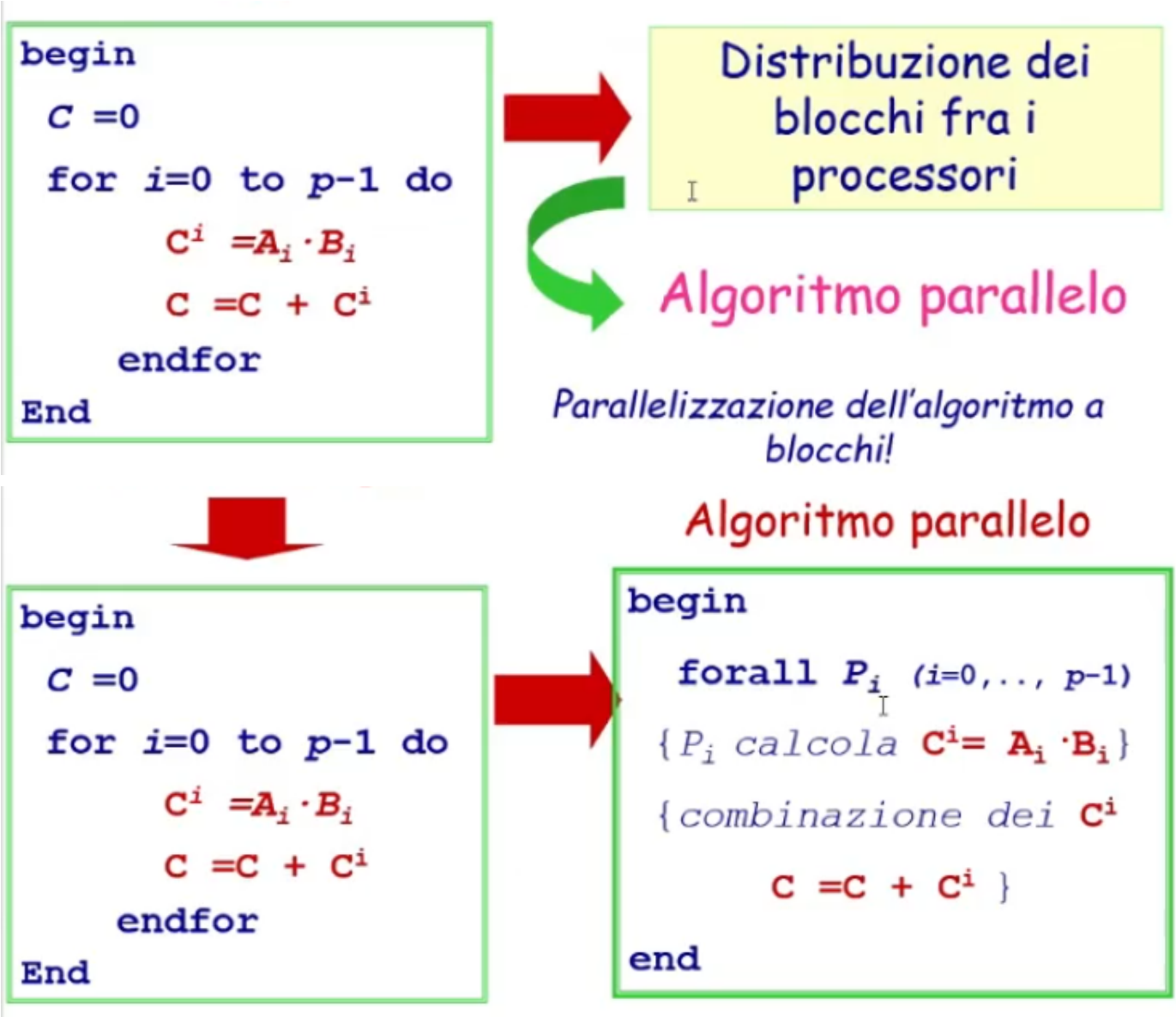
Risoluzione dei sottoproblemi

Nella seconda strategia il prodotto $AxB = C$ viene decomposto in *p* prodotti del tipo

$$A_i \ x \ B_i \ = \ C^i$$

dove

$$C = \sum_{i=0}^{p-1} C^i$$



Un'altra ancora può essere quella di organizzare la matrice C in blocchi quadrati e analogamente riorganizziamo in blocchi di colonne le matrici *A* e *B*

Decomponiamo il problema $\rightarrow$ le matrici *A* e *B* vengono distribuite in BLOCCHI di COLONNE fra *p* processori.
Nella terza strategia il prodotto $A\ x\ B = \ C$ viene decomposto in *p* x *p* prodotti del tipo

$$C_i = \sum_{k=0}^{p-1} A_k\ B_{ki}$$

Ciascun processore calcola *p* prodotti matrice-matrice (di dimensione più piccola di quello assegnato)

```
begin
 for i=0 to p-1 do
  C_i =0
  for j=0 to p-1 do
     C_i = C_i + A_j ·B_ji
  endfor
 endfor
end
```

Distribuzione dei blocchi fra i processori

Algoritmo parallelo

Parallelizzazione dell'algoritmo a blocchi!

Lezione del 21-12

Lezione del 21-12

Prodotto matrice matrice

(prodotto BMR)

È necessario avere dimensioni coerenti

Gli elementi di C sono calcolati effettuando i prodotti scalari tra le righe della matrice A con le colonne della matrice B

Decomposizione in BLOCCHI di RIGHE + BLOCCHI di COLONNE = decomposizione in BLOCCHI QUADRATI

Al processo P_{ij} assegniamo i blocchi A_{ij} e B_{ij}

$C_{00} = A_{00}B_{00} + A_{01}B_{10} + A_{02}B_{20}$	$C_{01} = A_{00}B_{01} + A_{01}B_{11} + A_{02}B_{21}$	$C_{02} = A_{00}B_{02} + A_{01}B_{12} + A_{02}B_{22}$
$C_{10} = A_{10}B_{00} + A_{11}B_{10} + A_{12}B_{20}$	$C_{11} = A_{10}B_{01} + A_{11}B_{11} + A_{12}B_{21}$	$C_{12} = A_{10}B_{02} + A_{11}B_{12} + A_{12}B_{22}$
$C_{20} = A_{20}B_{00} + A_{21}B_{10} + A_{22}B_{20}$	$C_{21} = A_{20}B_{01} + A_{21}B_{11} + A_{22}B_{21}$	$C_{22} = A_{20}B_{02} + A_{21}B_{12} + A_{22}B_{22}$

II passo

A_{00} B_{00}

$C_{00} =$

$A_{00}\ B_{00} + \boxed{A_{01}}\ B_{10} + A_{02}\ B_{20}$

A_{00} A_{01} B_{01}

$C_{01} =$

$A_{00}\ B_{01} + \boxed{A_{01}}\ B_{11} + A_{02}\ B_{21}$

A_{00} A_{02} B_{02}

$C_{02} =$

$A_{00}\ B_{02} + \boxed{A_{01}}\ B_{12} + A_{02}\ B_{22}$

A_{11} A_{10} B_{10}

$C_{10} =$

$A_{10}\ B_{00} + A_{11}\ B_{10} + A_{12}\ B_{20}$

A_{11} B_{11}

$C_{11} =$

$A_{10}\ B_{01} + A_{11}\ B_{11} + A_{12}\ B_{21}$

A_{11} A_{12} B_{12}

$C_{12} =$

$A_{10}\ B_{02} + A_{11}\ B_{12} + A_{12}\ B_{22}$

A_{22} A_{20} B_{20}

$C_{20} =$

$A_{20}\ B_{00} + A_{21}\ B_{10} + A_{22}\ B_{20}$

A_{22} A_{21} B_{21}

$C_{21} =$

$A_{20}\ B_{01} + A_{21}\ B_{11} + A_{22}\ B_{21}$

A_{22} B_{22}

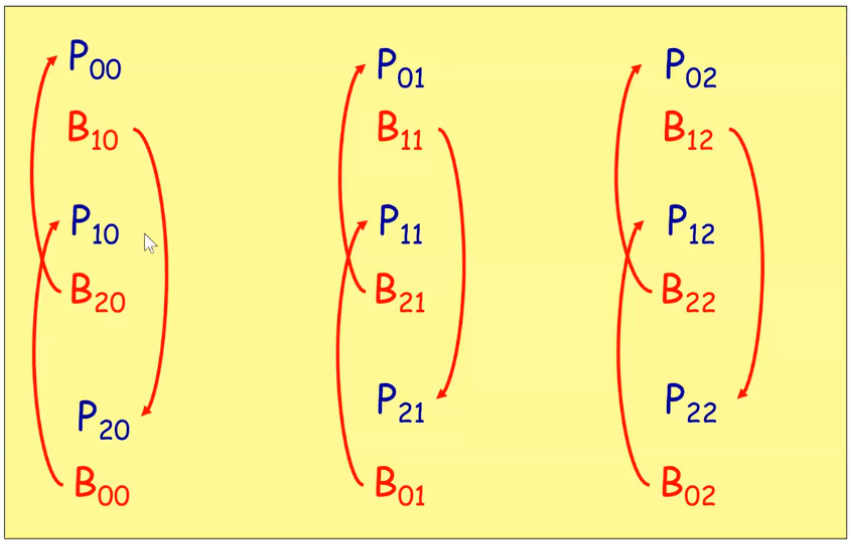
$C_{22} =$

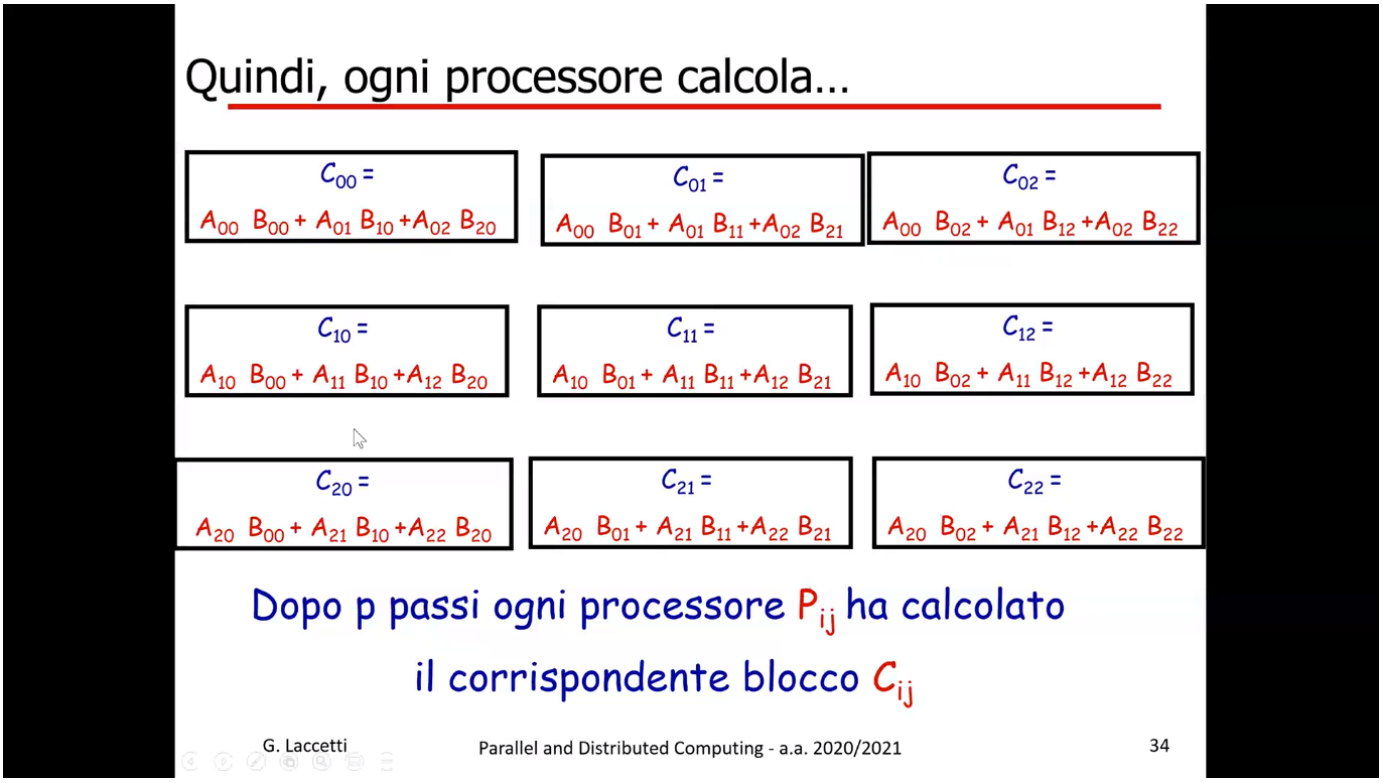
$A_{20}\ B_{02} + A_{21}\ B_{12} + A_{22}\ B_{22}$

Nel calcolo dei C_{ij} ...

- A_{01} è presente nel calcolo della I riga di C (C_{00}, C_{01}, C_{02})

Comunicazione dei blocchi di B





BMR → Broadcast Multiply Rolling → strategia costituita da p passi (con p numero di diagonali)

Questo tipo di strategia è utilizzabile solo nel caso di matrici quadrate. Si parte dalla diagonale principale della griglia di processori, ad ogni passo k, si considera la k-esima diagonale situata al di sopra di quella principale. I processori situati lungo la diagonale effettuano una comunicazione collettiva del blocco di A in loro possesso a tutti i processori della medesima riga. Inoltre ad ogni passo ciascun processore effettua una comunicazione del proprio blocco di B al processore situato nella stessa colonna e nella riga precedente